Maintenance Is Easier

One of the primary jobs of modules is to enforce logical boundaries between a number of different problem domains. If care is taken while designing and writing modules and if interdependency is minimized between different modules, then changes or modifications made to a single module do not need to have a necessary impact on other parts of the program. This could also mean that in extreme cases, a user could make changes only to a single module without even ever being aware of what the application looks like outside that single module. This feature makes it easy for relatively large teams of programmers to work in collaboration with each other on larger applications without having to know how the whole thing works.

Reusability is Higher

Modular programming will help eliminate the need for duplicate code. How, you ask? Functionality that is defined in a single module is easily available for reuse by one or many other parts of the application. All it requires is an appropriately defined interface. This definitely makes things easier.

Avoids Collisions

If you get your hands on PEP 20 - Zen of Python, it specifies that namespaces are a great idea. Modules will typically define a separate namespace. What this helps is to avoid collisions of any sort between the identifiers present in different areas of the same program.

Modules, packages and functions are all constructs in Python that are present to promote code modularisation.

Modules

So what is a Python module? Any or every file that has a file extension `.py` and consists of legit Python code can be considered as and is definitely a module. There is no special syntax that needs to be used to convert such a file into a module. A module is known to contain arbitrary objects like files, classes or attributes. There are a number of ways that one can import a module. Let's demonstrate using the math module -

- `import math`

The math module will provide mathematical constants as well as functions, for instance - the sine function (math.sin()) as well as the cosine function (math.cos()) or ℗ (math.pi). Every attribute or function can be accessed only when you add "math" in front of their names, as shown